Copyright All rights reserved

20. 프로그래밍 설계
(Programming Architecture)

Heesung Oh
https://github.com/3dapi

20 프로그래밍 설계

Heesung Oh
Copyright All rights reserved

- 구조 설계와 모듈화
  - 프로그램을 "기능"이 아니라 "구조"로 설계
  - 코드 작성 전에 실행 흐름을 먼저 정의

- 탑다운 설계 (Top-Down Design)
  - 요구 사항 정의 → 전체 구조 설계→ 모듈 설계→ 세부 구현
  - 구현보다 구조가 먼저

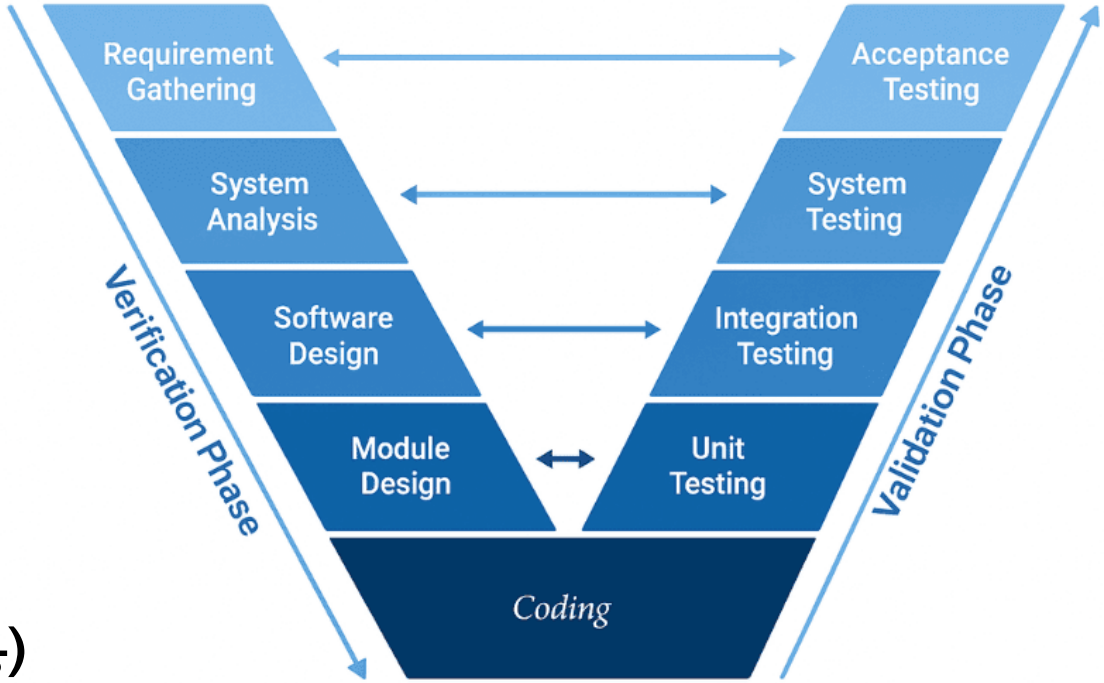
20 V 모델

Heesung Oh
Copyright All rights reserved

- V 모델 (V-Model)
  - 설계 단계 ↔ 테스트 단계 대응
  - 구조와 검증을 함께 고려

- 설계 단계 (왼쪽)
  - 요구사항 정의
  - 전체 구조 설계
  - 모듈 설계
  - 세부 구현

- 검증 단계 (오른쪽)
  - 단위 테스트 (함수/모듈)
  - 통합 테스트 (모듈 간 연동)
  - 시스템 테스트 (전체 프로그램)



20.1 코드 구조와 설계 원칙

Heesung Oh
Copyright All rights reserved

- 코드 구조와 설계 원칙
  - 기능 구현 능력 ≠ 구조 설계 능력
  - 구조 설계는 프로그램의 전체 흐름과 역할 분담을 정의
- 기능 중심 설계
  - 기능을 하나씩 구현
  - 빠른 개발 가능
  - 규모 증가 시 복잡도 증가
  - 수정 영향 범위 예측 어려움
- 구조 중심 설계
  - 실행 흐름과 책임을 먼저 정의
  - 구조 안에 기능 배치
  - 확장 및 유지보수에 유리

Heesung Oh
Copyright All rights reserved

프로그램 규모와 구조 필요성

- 작은 프로그램
  - 단일 파일 가능
  - 흐름 단순
  - 수정 영향 범위 제한적
- 큰 프로그램 문제
  - 코드 집중
  - 기능 간 영향 증가
  - 흐름 파악 어려움

20.2 모듈화와 파일 분리

Heesung Oh
Copyright All rights reserved

- .c 와 .h 역할
  - .c → 구현 (Implementation)
  - .h → 인터페이스 (Interface)

- 구성 원칙
  - 함수 선언 → .h
  - 함수 정의 → .c
  - 외부에 필요한 것만 노출
  - 내부 구현은 숨김

- 인터페이스가 유지되면
  구현 변경이 다른 모듈에 영향 없음

20.2 모듈화와 파일 분리

Heesung Oh
Copyright All rights reserved

- 인터페이스 분리와 의존성
  - 인터페이스 vs 구현
    - 인터페이스 → 무엇을 제공하는가
    - 구현 → 어떻게 동작하는가
  - 장점
    - 구현 변경 용이
    - 모듈 간 의존성 감소
    - 테스트 용이
- include 의존성 관리
  - 필요한 선언만 포함
  - 내부 구현은 .c에 위치
  - 순환 참조 방지
- 주의
  - 헤더 파일은 중복 포함 방지 필요
  - → #pragma once 또는 include guard

20.2 모듈화와 파일 분리

Heesung Oh
Copyright All rights reserved

- 시스템 구조로 적용: 모듈 분리 적용
  - Input → 사용자 입력 처리
  - Data → 점수, 기록, 설정 관리
  - Output → 결과 출력
  - GameLogic → 게임 규칙 처리
  - RunFlow → 전체 실행 흐름 제어

- 정리
  - 기능을 시스템 단위로 분리
  - 각 모듈은 독립적으로 설계
  - RunFlow가 전체 구조를 제어